

Chapter – 4

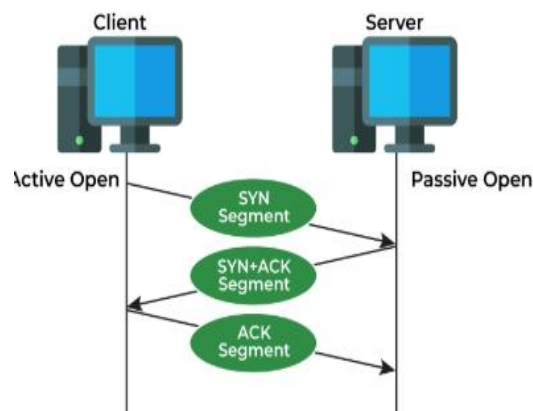
Distributed Network Programming

The term network programming refer to writing programs that executes across multiple devices (computers), in which the devices are all connected to each other using a network. It is also referred as socket programming because we use the concept of socket to write programs that can communicate in the network. The java.net package of the J2SE APIs contain a collection of classes and interfaces that provides the low – level communication details, allowing us to write programs that focus on solving the problem at hand. The java.net package provides support for the two common network protocols: TCP and UDP. These protocols lie at transport layer of TCP/IP protocol suit.

When we write java programs that communicate over the network, we do programming at the application layer. We don't need to concern ourselves with TCP and UDP layers. Instead, we can use the classes in the java.net package. These classes provide system – independent network communication. However, to decide which java classes your programs should use, we need to understand how TCP and UDP differ.

Transmission Control Protocol(TCP)

Transmission Control Protocol (TCP) is one of the main protocols of the internet protocol suite. It lies between the Application and Network Layers which are used in providing reliable delivery services. It is a connection – oriented protocol for communications that helps in the exchange of messages between different devices over a network. The Internet Protocol (IP), which establishes the technique for sending data packets between computers, works with TCP.

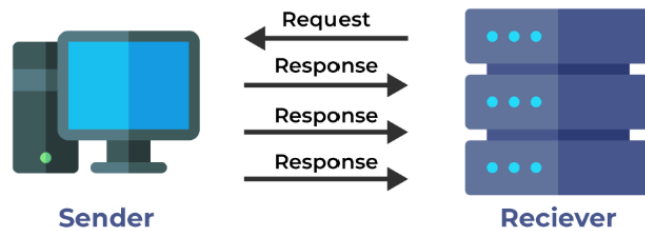


Features of TCP

- TCP keep track of the segments being transmitted or received by assigning numbers to every single one of them.
- Flow control limits the rate at which a sender transfer data. This is done to ensure reliable delivery.
- TCP implements an error control mechanism for reliable data transfer.
- TCP takes into account the level of congestion in the network.

User Datagram Protocol (UDP)

User Datagram Protocol is a transport layer protocol. UDP is a part of the internet Protocol suite, referred to as the UDP/IP suite. Unlike TCP, it is an unreliable and connectionless protocol. So there is no need to establish a connection before data transfer. The UDP helps to establish low – latency and loss – tolerating connections establish over the network. The UDP enables process to process communication.



Feature of UDP

- Used for simple request – response communication when the size of data is less and hence there is lesser concern about flow and error control.
- It is a suitable protocol for multicasting as UDP supports packet switching.
- UDP is used for some routing update protocols like RIP(Routing Information Protocol).
- Normally used for real – time applications which can not tolerate uneven delays between sections of a received message.

TCP Vs UDP

- TCP is a connection – oriented protocol, whereas UDP is a connectionless protocol.
- The speed for TCP is slower while the speed of UDP is faster.
- TCP uses handshake protocol while UDP uses no handshake protocol.
- TCP does error checking and also makes error recovery, on the other hand, UDP performs error checking, but discards packets having error.
- TCP has acknowledgement segments, but UDP does not have.
- TCP is heavy – weight and UDP is lightweight.

When to use UDP and TCP?

- TCP is an ideal choice, when most of the overhead is in the connection. Use TCP sockets when both client and server independently send packets. Example: online poker use TCP when occasional delay is acceptable.
- UDP is an ideal to use with multimedia like VoIP. We should use UDP of both client and server may separately send packets. Example: multiplayer games. Use UDP when occasional delay is not acceptable.

IP Address

An IP address (Internet Protocol address) is a unique identifier assigned to each device connected to a network that uses the Internet Protocol for communication and prevents address conflicts. IP addresses are fundamental in cloud architectures, enabling virtual machines, containers and services to communicate within and across cloud platforms.

By using IP addressing schemes network can be segmented into subnets, improving security, performance and manageability. IP addresses are integral to firewall rules, access control lists (ACLs) and intrusion detection system which help to enforce security policies and protect against unauthorized access. IP address consists of two parts IPV4 and IPV6.

IP address is an essential part as it allows the service provider to identify networks and their address provide information about the network and data being shared. IP address is stored on every computer that is connected to the internet and this helps their communication over LAN and internet.

Port Number

A port number is like a specific address that helps direct messages to the right program or service on a server. When a message or request is sent over the internet, the port number tells the server which program should handle the message. Different port numbers are used to determine what protocol incoming traffic should be directed to. Ports are identified for each protocol and it is considered as a communication end point.

For example, websites use port 80 for regular web traffic. So when you visit a website, your message goes to port 80 on the server, where the web service (HTTP) listens and responds with the webpage. Different services use different port numbers and each port is assigned to a specific function, like port 25 for email (SMTP) or port 443 for secure websites (HTTPS). This system ensures that the right program gets the right message. Ports are represented by 16-bit numbers. There are 2^{16} port numbers i.e. 65536.

NETWORK PORT

Well-Known Ports	0-1023
Registered Ports	1024-4915
Dynamic Port	49152-65535

Networking Classes in the JDK

Java programs can use TCP and UDP to communicate over the internet. Some classes provided in the `java.net` package are:

Socket Class: The java socket class is used to create sockets when we use TCP for communication.

Server Socket Class: The ServerSocket class is used to create socket for servers when TCP is used for communication.

InetAddress Class: The InetAddress is java's representation of an IP address. Instances of this class are used together with UDP DatagramSocket and TCP.

DatagramSocket Class: This class is used to create sockets at client or server when we use UDP protocol for communication.

DatagramPacket Class: This class is used to create datagram packets that are exchanged between UDP clients and UDP servers.

URL Class: The java URL class represents an URL. URL is an acronym for Uniform Resource Locator. It points to a resource on the World Wide Web.

URLConnection Class: The java URLConnection class represents a communication link between the URL and the application. This class can be used to read and write data to the specified resource referred by the URL.

Working with URLs

URL is the acronym for Uniform Resource Locator. It is simply a string of text that identifies all the resources on the internet, telling us the address of the resource, how to communicate with it, and retrieve something from it.

<http://www.youtube.com/how-to-design-a-tiny-url>

A URL contains many information such as

1. Protocol: In above example, http is the protocol.
2. Server name or IP address: in above example www.youtube.com is the server name.
3. Port Number: It is an optional attribute. It can be specified by writing after domain name as: <http://www.youtube.com:80/how-to-design-a-tiny-url>, 80 is the port number.
4. File Name or directory name: In this case, [how-to-design-a-tiny-url](http://www.youtube.com/how-to-design-a-tiny-url) is the file name.

URL Class

The URL class in java is a fundamental component for accessing resources on the internet. The URL can point to various types of resources such as static file, dynamic content, API's. The URL class provides constructors and methods to create URL objects and retrieve their components.

Constructors of URL class

SN	Constructors	Description
1	URL(String address)	It creates a URL object from the specified string.
2	URL(String protocol, String host, String file)	Creates a URL object from the specified protocol, host and file name.
3	URL(String protocol, String host, int port, String file)	Creates a URL object from the specified protocol, host, port and file name.
4	URL(URL context, String spec)	Creates a URL object by parsing the given spec in the given context.
5	URL(String protocol, String host, int port, String file, URLStreamHandler handler)	Creates a URL object from the specified protocol, host, port, file name and handler.

Methods of URL class

SN	Method	Description
1	Public String getProtocol()	It returns the protocol of the URL.
2	Public String getHost()	It returns the host name of the URL.
3	Public String getPort()	It returns the port number of the URL.
4	Public String getFile()	It returns the file name of the URL.
5	Public String getPath()	It returns the path of the URL.
6	Public String getQuery()	It returns the query part of the URL.
7	Public URLConnection openConnection()	It returns the instance of URLConnection associated with this URL.

Example:

```
import java.net.*;
import java.io.*;

public class urlClass
{
    public static void main(String[] args)
    {
        try
        {
            URL url = new URL("https://192.168.1.5:/index.html");

            System.out.println("URL is " + url.toString());
            System.out.println("Protocol is " + url.getProtocol());
            System.out.println("Authority is " + url.getAuthority());
            System.out.println("File name is " + url.getFile());
            System.out.println("Host is " + url.getHost());
            System.out.println("Path is " + url.getPath());
            System.out.println("Port is " + url.getPort());
        }
        catch (MalformedURLException e)
        {
            System.out.println(e);
        }
    }
}
```

Output:

URL is https://192.168.1.5:/index.html

Protocol is https

Authority is 192.168.1.5:

File name is /index.html

Host is 192.168.1.5

Path is /index.html

Port is -1

URLConnection Class

The java URLConnection class represents a communication link between the URL and the application. This class can be used to read and write data to the specified resource referred by the URL. It is not possible to instantiate the URLConnection class directly. Instead we have to create the URLConnection via the openConnection() method of URL class.

Syntax for URLConnection class is

Public URLConnection openConnection() throws IOException

```
{  
}
```

Example

```
import java.net.*;  
import java.io.*;  
  
public class urlConnection  
{  
    public static void main(String[] args)  
    {  
        try  
        {  
            URL url = new URL("https://pu.edu.np/");  
            URLConnection urlConnection = url.openConnection();  
            InputStream inputStream = urlConnection.getInputStream();  
  
            int i;  
            while ((i = inputStream.read()) != -1)  
            {  
                System.out.print((char) i);  
            }  
  
            inputStream.close();  
        }  
        catch (Exception e)  
        {  
            System.out.println(e);  
        }  
    }  
}
```

InetAddress class

Java InetAddress class represents an IP address. The java.net.InetAddress class provides methods to get the IP of any host name for example www.pu.edu.np, www.google.com, www.facebook.com etc.

Commonly used method of InetAddress class are

SN	Method	Description
1	getByName(String host)	It is static method and returns the instance of InetAddress containing local Host IP and name
2	getLocalHost()	It returns the instance of InetAddress containing local host name and address.
3	getHostName()	It returns the host name of the IP address.
4	getHostAddress()	It returns the IP address in string format.
5	getAllByName(host)	It returns an array of IP's about the host.

Example

```
import java.net.*;

class Inetadd
{
    public static void main(String args[])
    {
        try
        {
            InetAddress IP = InetAddress.getByName("pu.edu.np");

            System.out.println("Host address: " + IP.getHostAddress());
            System.out.println("Host Name: " + IP.getHostName());
        }
        catch (Exception e)
        {
            System.out.println("Unknown host or invalid address: " + e.getMessage());
        }
    }
}
```

Output

Host address: 172.67.222.102

Host Name: pu.edu.np

Socket Programming Using TCP

The two key classes from the java.net package used in creation of server and client programme are Socket and ServerSocket.

Steps for writing client program

1. Open a Socket

```
Socket client = new Socket(server_address, port_id)
```

The statement creates a socket for client computer. While creating socket we need to provide IP address or domain name of the computer where server program executes and the port number in which server listens to the client.

2. Open an input stream and output stream to the socket

```
Scanner ins = new Scanner(client.getInputStream());  
PrintWriter outs = new PrintWriter(client.getOutputStream(), true);
```

The first statement gets input stream of client's socket and opens Scanner on it. Similarly, the second statement gets output stream of client's socket and opens PrintWriter on it.

3. Read from and write to the sockets stream

```
rs = ins.nextLine();  
outs.println(ss);
```

The first statement reads a line from client's socket and puts it in variable 'rs' and the second statement write the value of string variable 'ss' into client's socket.

4. Close the stream

```
ins.close();  
outs.close();
```

Once I/O is completed, above statements are used to close input stream and output stream of client's socket.

5. Close the socket

```
client.close();
```

We use this statement to close the connection between client's and server computer.

Steps for writing Server Program

1. Create the Object of ServerSocket class

```
ServerSocket server = new ServerSocket(2345);
```

This statement create socket for the server and the argument represents the port number to which the server listens.

2. Accept the connection from the client

```
Socket client = server.accept()
```

After creating the socket it enters into the infinite loop and waits for the client connection by using accept method.

3. Get input and output streams of the Socket

```
Scanner ins = new Scanner(server.getInputStream());
```

```
PrintWriter outs = new PrintWriter(server.getOutputStream(), true);
```

The first statement gets input stream of server's socket and opens Scanner on it. Similarly, the second statement gets output stream of server's socket and opens PrintWriter on it.

4. Read/Write Data to the Socket

```
rs = ins.nextLine();
```

```
outs.println(ss);
```

The first statement read a line from server's socket and puts it in variable 'rs' and the second statement write the value of string variable 'ss' into server's socket.

5. Close Stream

```
ins.close();
```

```
outs.close();
```

Once I/O is completed, above statements are used to close input stream and output stream of client's socket.

6. Close Socket

```
server.close();
```

```
client.close();
```

We use these statements to close the connection between client and server computers.

Example

```
// Client program
package network.programming;
import java.net.*;
import java.io.*;
import java.util.*;

public class tcpClient
{
    public static void main(String a[]) throws IOException
    {
        // Open your connection to a server, at port 1250
        Socket cs = new Socket("localhost", 1250);

        // Get an input file handle from the socket and read the input
        Scanner ins = new Scanner(cs.getInputStream());
        PrintWriter outs = new PrintWriter(cs.getOutputStream(), true);

        // Perform IO
        outs.println("What are the courses offered by Pokhara University");
        String s = ins.nextLine();
        System.out.println("Data from Server: " + s);

        // Close streams and connection
        ins.close();
        outs.close();
        cs.close();
    }
}
```

```
// Server program
package network.programming;
import java.net.*;
import java.io.*;
import java.util.*;

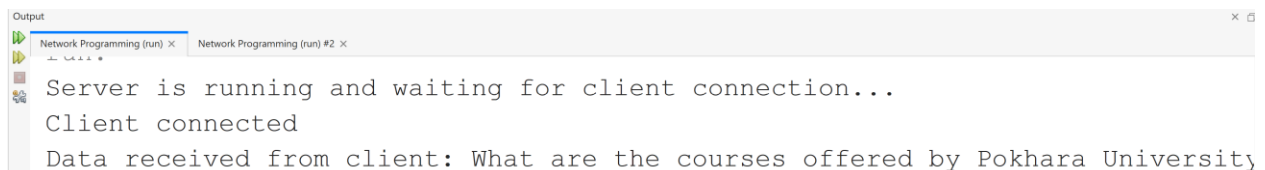
public class tcpServer
{
    public static void main(String a[]) throws IOException
    {
        // Register service on port 1250
        ServerSocket ss = new ServerSocket(1250);
        System.out.println("Server is running and waiting for client connection...");

        // Wait and accept a connection
        Socket cs = ss.accept();
        System.out.println("Client connected. Waiting for message from client...");

        // Get a communication stream associated with the socket
        Scanner ins = new Scanner(cs.getInputStream());
        PrintWriter outs = new PrintWriter(cs.getOutputStream(), true);

        // Perform I/O
        String s = ins.nextLine(); // This waits until client sends something
        System.out.println("Data received from client: " + s);
        outs.println("BE. Computer, BE. Software, BE.IT, BE.Civil, BCA, ME. Computer");

        // Close streams and connection
        outs.close();
        ins.close();
        cs.close();
        ss.close();
    }
}
```



```
Output
Network Programming (run) x Network Programming (run) #2 x
Server is running and waiting for client connection...
Client connected
Data received from client: What are the courses offered by Pokhara University
```



```
Output
Network Programming (run) x Network Programming (run) #2 x
run:
Data from Server: BE. Computer, BE. Software, BE.IT, BE.Civil, BCA, ME.
```

Socket Programming Using UDP

The UDP protocol provides a mode of network communication whereby applications send packets of data called datagram's to one another. Java supports datagram communication through the following classes:

- DatagramPacket
- DatagramSocket

There is no concept of connection in the UDP system. Hence transmission of data onto the network does not require any host to be listening.

Steps for writing client program

1. Get a datagram socket

```
DatagramSocket socket = new DatagramSocket(port);
```

This creates a socket for client without specifying port number. The server gets the port number from the DatagramPackets and sends its response to that port.

2. Send request

```
DatagramPacket packet = new DatagramPacket(byte[], length, inetaddr, port)  
socket.send(packet)
```

The statement creates a datagram packet to be sent by specifying empty byte array, length of data in bytes, InetAddress and port number. Empty byte array indicates that this is the request to server. The second statement sends the packet to the server. After receiving this packet, server will send all message to the port number specified in the packet.

3. Get response

```
Packet = new DatagramPacket(buf, buf.length);  
Socket.receive(packet);
```

The client creates a "receive" packet and uses the DatagramSocket receive method to receive the reply from the server. The receive method waits until a datagram packet destined for the client comes through the socket. Normally, a client sets a timer so that it does not wait forever for a reply.

4. Display response

```
String s = new String(packet.getData());  
System.out.println("From Server:" +s);
```

When the client receives a reply from the server, the client uses the getData method to retrieve that data from the packet. The client then converts the data to a string and display it.

5. Close Socket

```
socket.close();
```

This statement closes the datagram socket that is created for communication between client and server.

Steps for writing Server program

1. Get a datagram socket

```
DatagramSocket socket = new DatagramSocket(port);
```

This instantiates DatagramSocket with the port number specified in parameter. This parameter is the UDP port from which the server receive UDP packets.

2. Receive request

```
byte[] buf = new byte[256];
```

```
DatagramPacket packet = new DatagramPacket(buf, buf.length);
```

```
socket.receive(packet);
```

This first statement simply creates a byte array and second statement create a datagram packet for receiving the packet sent by client and finally third statement receives the datagram packet.

3. Send response

```
String s = new String ("Hello... .. UDP client");
```

```
buf = s.getBytes();
```

```
packet = new DatagramPacket(buf, buf.length, address, port);
```

```
socket.send(packet);
```

The first and second statement prepares data for sending. Third statement instantiates DatagramPacket class by including the data prepared in above step and finally third statement sends the packet to the client.

4. Close Socket

```
socket.close();
```

This statement closes the datagram socket that is created for communication between client and server.

Example

//UDP Client Program

```
package network.programming;

import java.net.*;
import java.io.*;

public class UDPClient
{
    public static void main(String[] args) throws IOException
    {
        // Create socket
        DatagramSocket socket = new DatagramSocket();

        // Prepare request data
        byte[] buf = new byte[256]; // Sending empty buffer as placeholder
        InetAddress address = InetAddress.getByName("localhost");

        // Send request packet to the server
        DatagramPacket packet = new DatagramPacket(buf, buf.length, address, 4445);
        socket.send(packet);
        System.out.println("Request sent to server.");

        // Prepare to receive the response
        packet = new DatagramPacket(buf, buf.length);
        socket.receive(packet);

        // Convert response to string properly (using packet.getLength())
        String received = new String(packet.getData());
        System.out.println("Quote of the Moment: " + received);

        // Close the socket
        socket.close();
    }
}
```

//UDP Server

```
package network.programming;
```

```
import java.net.*;
```

```
import java.io.*;
```

```
public class UDPServer
```

```
{
```

```
    public static void main(String[] args) throws IOException
```

```
    {
```

```
        // Buffer for receiving incoming data
```

```
        byte[] buf = new byte[256];
```

```
        // Create a datagram socket and bind it to port 4445
```

```
        DatagramSocket socket = new DatagramSocket(4445);
```

```
        System.out.println("UDP Server is running and waiting for a packet...");
```

```
        // Receive request
```

```
        DatagramPacket packet = new DatagramPacket(buf, buf.length);
```

```
        socket.receive(packet); // Blocking call
```

```
        // Extract client info
```

```
        InetAddress address = packet.getAddress();
```

```
        int port = packet.getPort();
```

```
        System.out.println("Received request from " + address + ":" + port);
```

```
        // Send a response back to the client
```

```
        String response = "Hello";
```

```
        buf = response.getBytes();
```

```
        packet = new DatagramPacket(buf, buf.length, address, port);
```

```
        socket.send(packet);
```

```
        System.out.println("Sent response: " + response);
```

```
        // Close socket
```

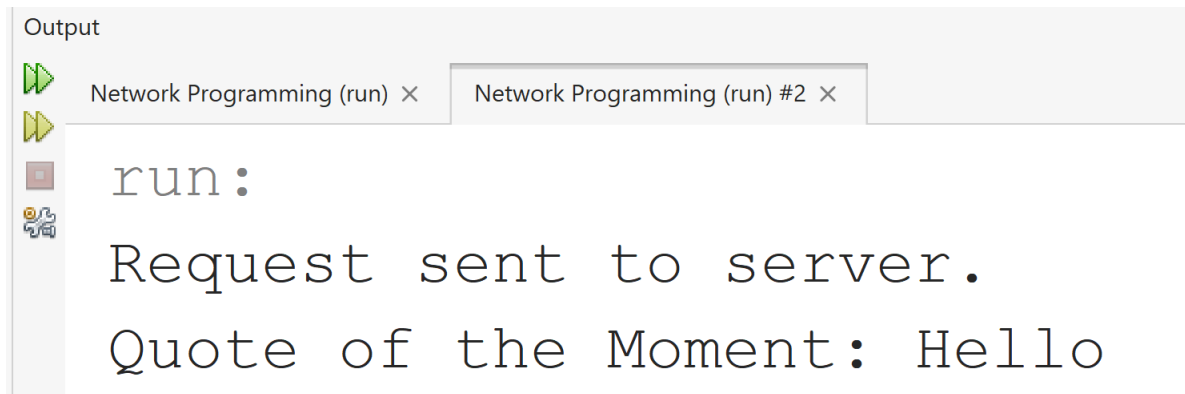
```
        socket.close();
```

```
        System.out.println("Server socket closed.");
```

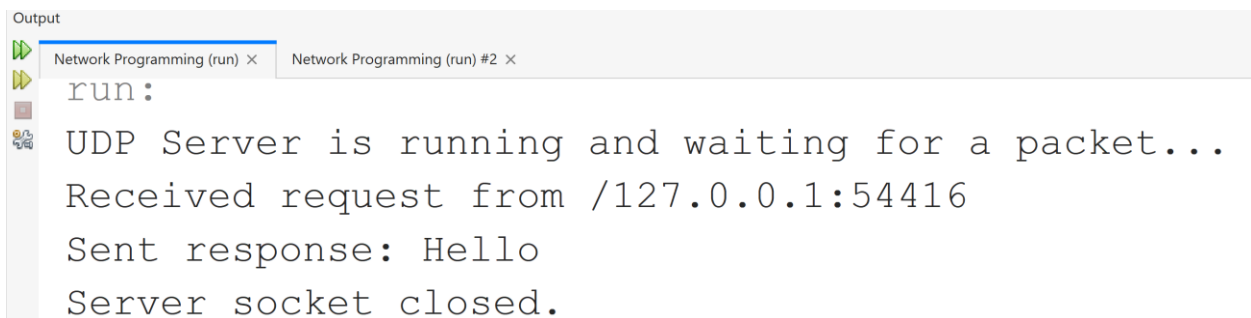
```
    }
```

```
}
```


Output



```
run:
Request sent to server.
Quote of the Moment: Hello
```



```
run:
UDP Server is running and waiting for a packet...
Received request from /127.0.0.1:54416
Sent response: Hello
Server socket closed.
```

Java Mail API

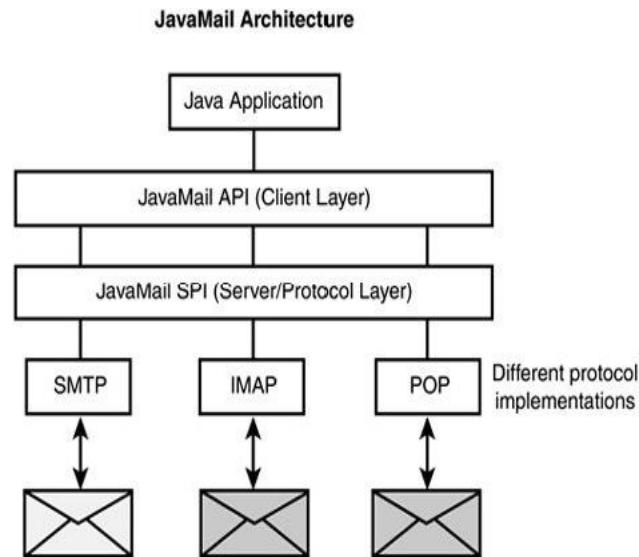
The javaMail is an API that is used to compose, write and read electronic message(emails). The javaMail API provides protocol – independent and platform – independent framework for sending and receiving mails. The javax.mail and javax.mail.activation packages contains the core classes of JavaMail API. The javaMail facility can be applied to many events. It can be used at the time of registering the user (for sending notification such as thanks for your internet to my site), forget password(sending password to the users email id), sending notifications for important updates etc.

If we use java 2 platform, Enterprise Edition(J2EE) 1.3, it includes JavaMail, so no additional setup is required. If, however we are running java 2 platform, Standard Edition(J2SE) 1.1.7 and upward, and we want email capability for our applications, we need to download and install the following

- JavaMail API
- Java Activation Framework

Java Mail Architecture

Java application uses JavaMail API to compose, send and receive emails. The following figure illustrate the architecture of JavaMail:



The abstract mechanism of JavaMail API is similar to other APIs, such as JDBC, JNDI etc. As seen the architecture diagram above, JavaMail API is divided into two main parts:

- **An application – independent part**

An application – programming interface (API) is used by the application components to send and receive mail message, independent of the underlying provider or protocol used.

- **A service – dependent part**

A service provider interface (SPI) speaks the protocol – specific language, such as SMTP, POP, IMAP and Network News Transfer Protocol (NNTP). It is used to plug in a provider of an email – service to the java platform.

Example

```
import java.util.Properties;
import javax.mail.*;
import javax.mail.internet.*;
import javax.activation.*;

public class mailAPI extends Authenticator
{
    static String from = "nishanta.212915@ncit.edu.np";
    static String pass = "nishanta@123";
    static InternetAddress to;

    public static void main(String[] args) throws MessagingException
    {
        //Get the session object
        Properties prop = System.getProperties();
        prop.put("mail.smtp.auth", "true");
        prop.put("mail.smtp.starttls.enable", "true"); //starts TLS(Transport Layer Security)
        prop.put("mail.smtp.host", "smtp.gmail.com");
        prop.put("mail.smtp.port", "587");
        prop.put("mail.smtp.user", "from");
        prop.put("mail.smtp.password", "pass");
        Session session = Session.getDefaultInstance(prop);

        //Compose the message
        MimeMessage msg = new MimeMessage(session);
        msg.setFrom(new InternetAddress("nishanta.212915@ncit.edu.np"));
        to = new InternetAddress("snishanta45@gmail.com");
        msg.addRecipient(Message.RecipientType.TO, to);
        msg.setSubject("Test java Mail");
        msg.setText("Hello, this is example of sending email");

        // Send email
        Transport trans = session.getTransport("smtp");
        trans.connect("smtp.gmail.com", from, pass);
        trans.sendMessage(msg, msg.getAllRecipients());
        System.out.println("Email sent successfully!");
    }
}
```

Remote Method Invocation

- Java Remote Method Invocation (RMI) is an object – oriented Remote Procedure Call (RPC) techniques. It allows us to invoke methods on an object that exists in a different address space.
- This address space may exist on the same computer or even on a different computer connected to the source computer by a network.
- So, it enable objects, distributed in different computers, to communicate with one another. Although it provides a simple but elegant model, it is remarkably powerful and can be used to invoke remote methods easily and in a natural way.

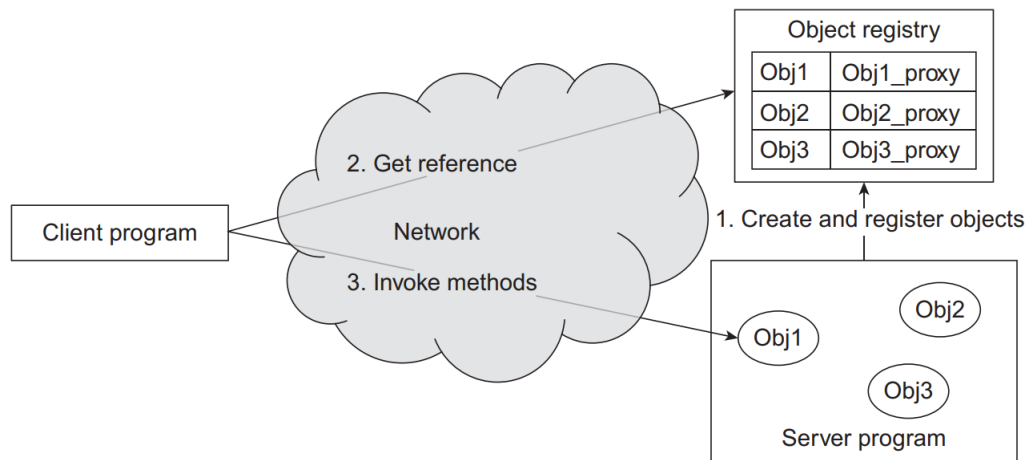


Fig: RMI programming model

- The underlying communication between clients and server in this model takes place seamlessly using sockets. It means that the message from the client is not method invocation in the OO sense. Instead, it is a stream of data that must be interpreted by the server before it can invoke method on the target object.
- In general, RMI architecture was developed taking the following goals into consideration
 - A primary goal of java RMI technology was to allow programmers to develop distributed java programs with the same syntax and semantics used for non – distributes programs.
 - Another goal was to create a distributed object model that fits the java programming language and the local object model naturally. RMI architects finally succeeded in creating such a powerful system that extends the safety and robustness of the java architecture to the distributed computing world.

Application Components

Three entities are often involved in an RMI application: a server program, a client program and object registry.

Server

This is a program that typically creates a remote object to be used for method invocation. This object is an ordinary object except that its class implements a Java RMI interface. Upon creation, the object is exported and registered with a separate application called object registry.

Client

A client program typically consults the object registry to get a reference (handle) to a remote object with a specified name. It can then invoke methods on the remote object using this reference (handle) as if the object is stored in the client's own address space. The RMI handles the details of communication (using sockets) between the client and the server and passes information back and forth. Note that this complex communication procedure is absolutely hidden to the client and server applications.

Object Registry

It is essentially a table of objects. Each entry of the table maps the object name to its proxy known as stub. The server registers the stub by name to the object registry. Once the stub is registered to the object registry successfully, the object is said to be available for other's use. Clients can now get a reference (handle) to the remote object (actually stub) from this registry and can invoke methods on it.

Goals of RMI

- To minimize the complexity of the application.
- To preserve type safety.
- Distributed garbage collection.
- Minimize the difference between working with local and remote objects.

RMI architecture

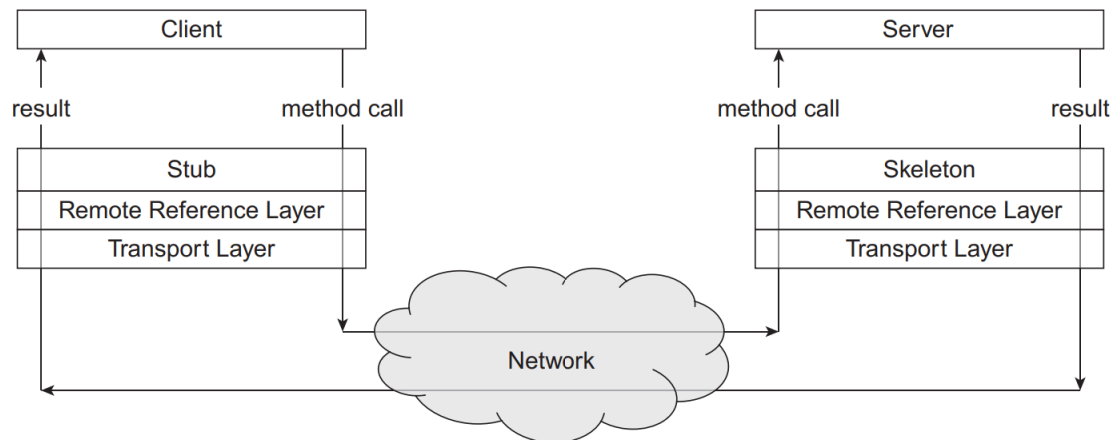


Fig: RMI Architecture

In an RMI application, we write two programs, a server program and a client program. Inside the server program, a remote object is created and reference of that object is made available for the client. The client program requests the remote objects on the server and tries to invoke its methods.

Application Layer

In this layer, client and server are involved in communication. The Java program on the client side communicates with the Java program on the server side.

Stub

The stub is an object that acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes a method on the stub object, it does the following tasks:

- It initiates a connection with remote Virtual Machines (JVM).
- It writes and transmits the parameter to remote Virtual Machine.
- It waits for the result.
- It reads the return value.
- Finally returns the value to the caller.

Skeleton

The skeleton is an object that acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming requests, it does the following tasks:

- It reads the parameter for the remote method.
- It involves the method on the actual remote object.
- It writes and transmits the result to the caller.

Remote Reference Layer (RRL)

The remote reference layer defines and supports the invocation semantics of the RMI connection. This layer maintains the session during the method call. This layer provides a **RemoteRef** object that represents the link to the remote service implementation objects. The stub objects use the **invoke()** method in **RemoteRef** to forward the method call. The **RemoteRef** object understands the invocation semantics for remote service.

Transport layer

It is responsible for setting up communication between two JVMs. All connections are stream – based network connections that use TCP/IP. Even if two JVMs are running on the same physical computer, they connect through their hosts computer's TCP/IP network protocol stack.

Steps of writing RMI application

Step 1: Create the remote interface

For creating the remote interface, extend the Remote interface and declare the RemoteException with all the methods of the remote interface.

Step 2: Provide the implementation of the Remote Interface

There are two ways for the implementation of the remote interface

- i. By writing implementation class separately
- ii. By directly make the server program implement the interface.

For providing the implementation of the Remote interface, we need to

- Either extend the UnicastRemoteObject class
- Or use the exportObject() method of the UnicastRemoteObject class.

Step 3: Create, Define and Run Server application

Create a client class from where to invoke the remote object. Create a remote object. Get the RMI registry using the getRegistry() method of the LocateRegistry class which belongs to the package java.rmi.registry. Bind the remote object created to the registry using the Rebind() method of the class named Registry.

Step 4: Create, Define and Run Client application

Create a client class. Get the RMI registry using the getRegistry() method of the LocateRegistry class which belongs to the package java.rmi.registry. Fetch the object from the registry using the method lookup() of the class Registry which belongs to the package java.rmi.registry. Invoke the required method using the obtained remote object.

Example

Adder.java

```
package rmi;
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Adder extends Remote
{
    int add(int n1, int n2) throws RemoteException;
}
```

RMIClient.java

```
import java.rmi.NotBoundException;
import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;

public class RMIClient
{
    public static void main(String[] args) throws RemoteException
    {
        RMIClient rc = new RMIClient();
        rc.connectRemote();
    }

    private void connectRemote() throws RemoteException
    {
        try
        {
            Scanner sc = new Scanner(System.in);
            Registry reg = LocateRegistry.getRegistry("localhost", 9999);
            Adder ad = (Adder) reg.lookup("hi server");
            System.out.println("Enter two numbers");
            int a = sc.nextInt();
            int b = sc.nextInt();
            System.out.println("Addition is " + ad.add(a, b));
        }
    }
}
```

```

        catch (RemoteException | NotBoundException e)
        {
            System.out.println("Exception: " + e);
        }
    }
}

```

RMIServer.java

```

import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.server.UnicastRemoteObject;

public class RMIServer extends UnicastRemoteObject implements Adder
{
    public RMIServer() throws RemoteException
    {
        super();
    }

    @Override
    public int add(int n1, int n2) throws RemoteException
    {
        return n1 + n2;
    }

    public static void main(String[] args)
    {
        try
        {
            Registry reg = LocateRegistry.createRegistry(9999);
            reg.rebind("hi server", new RMIServer());
            System.out.println("Server is ready");
        }

        catch (RemoteException e)
        {
            System.out.println("Exception: " + e);
        }
    }
}

```

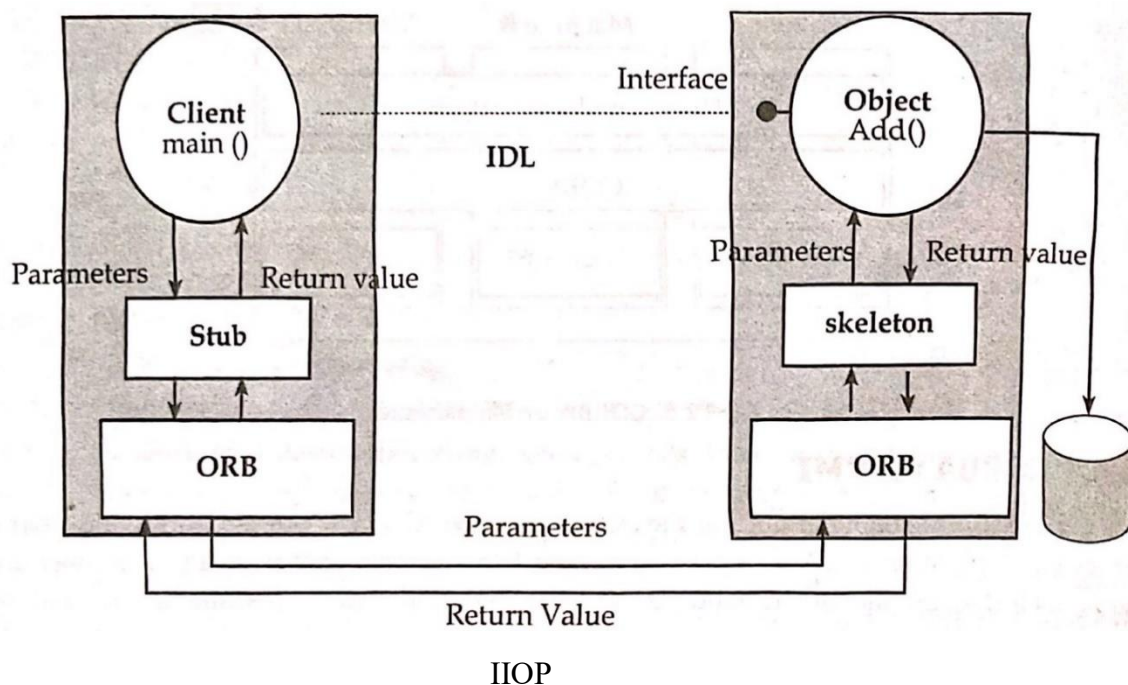
CORBA

- The Common Object Request Broker Architecture (CORBA) is a standard developed by the Object Management Group (OMG) to provide interoperability among distributed objects.
- CORBA is the world's leading middleware solution enabling the exchange of information, independent of hardware platforms, programming languages and operating systems.
- CORBA is often described as a “software bus” because it is a software – based communications interface through which objects are located and accessed.

RMI vs CORBA

RMI	CORBA
RMI is a Java Specific technology	CORBA is independent of programming language.
It uses java interface for implementation.	It uses Interface Definition Language(IDL) to separate interface from implementation.
RMI programs can download new classes from remote JVMs.	CORBA does not have this code sharing mechanism.
RMI passes objects by remote reference or by value.	CORBA passes objects by reference.
Distributed garbage collection is available.	Distributed garbage collection is not available.
Generally simpler to use.	More complicated.
It is free of cost.	Cost money according to vendor.

CORBA Architecture



The CORBA architecture consists of following components

Interface Definition Language (IDL)

CORBA uses the concept of an interface Definition Language to implement the services. IDL is a neutral programming language method of defining how a service is implemented. The syntax of this language is independent of any programming language. IDL is used to define an interface, which describes the methods that are available and the parameters that are required when a call is made for a remote object.

Object Request Broker (ORB)

The foundation of CORBA architecture is based on the Object Request Broker (ORB). The ORB is prime mechanism that acts as a middleware between the client and server. It provides a heterogeneous environment in which client can remotely communicate with the server irrespective of the hardware, the operating system, and the software development language used. The ORB helps to establish connection with remote server. When a client wants to use the services of a server, it first needs reference to the object that is providing the service.

Portable Object Adapter (POA)

The POA connects the server object implementation to the ORB. It extends the functionality of the ORB and some its services include activation and deactivation of the object implementations, generation and management of object references, mapping of object reference to their implementations and dispatching of client requests to server objects through a skeleton.

Naming Service

Defines how CORBA objects can be looked up by a name. It is a Common Object Service (COS) and allows an object to be published using a symbolic names and allows client to obtain references to the object using a standard API. The CORBA naming service provides a naming structure for remote objects.

Internet Inter – ORB protocol (IIOP)

The CORBA standard includes specification for inter – ORB communicate protocols that transmit object requests between various ORBs running on the network. The protocols are independent of the particular ORB implementations running at either end. An ORB implemented in java can talk to an ORB implemented in C, as long as they are both compliant with the CORBA standard. The inter – ORB protocol delivers messages between two ORBs. These message might be method requests, return values, error messages etc. If you want two ORBs to talk, just make sure they both speak the same inter ORB protocol (IIOP). The Inter – ORB protocol (IIOP) is based on TCP/IP and so is extensively used on the Internet.

Example

① Write the IDL file

```
module AddModule
{
    interface Adder
    {
        long add (in long a, in long b)
    };
};
```

② Generate the stub and skeleton code.

compile the above code as below

idl -fall Adder.idl

③ Write Server Code

```
import AddModule.*;
import org.omg.CORBA.*;
import org.omg.portableServer.*;

public class AdderImpl extends AdderPOA
{
    public long add (long a, long b)
    {
        return a+b;
    }
}

public static void main (String args[])
{
    try
    {
        ORB orb = ORB.init (args, null);
        POA rootpoa = POAHelper.narrow (orb.resolve_initial_reference ("RootPOA"));
        AdderImpl adderImpl = new AdderImpl ();
        org.omg.CORBA.Object ref = rootpoa.servant_to_reference (adderImpl);
        Adder href = AdderHelper.narrow (ref);

        org.omg.CORBA.Object objRef = orb.resolve_initial_references
            ("NameService");
        NamingContextExt ncRef = NamingContextExtHelper.narrow (objRef);
        ncRef.rebind (ncRef.to_name ("Adder"), href);
        System.out.println ("Adder server Ready");
        orb.run ();
    }
    catch (Exception e)
    {
        System.err.println ("Error:" + e);
    }
}
```

(4) Write Client Code

```
import AddModule.*;
import org.omg.CORBA.*;
import org.omg.CosNaming.*;

public class AdderClient
{
    public static void main(String a[])
    {
        try
        {
            ORB orb = ORB.init(args, null)
            org.omg.CORBA.Object objRef = orb.resolve_initial_references
                ("NameService");
            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);
            Adder ad = AdderHelper.narrow(ncRef.resolve_str("Adder"));
            long result = ad.add(5, 10);
            System.out.println("Result from server:" + result)
        }
        catch (Exception e)
        {
            System.err.println("Error:" + e);
        }
    }
}
```